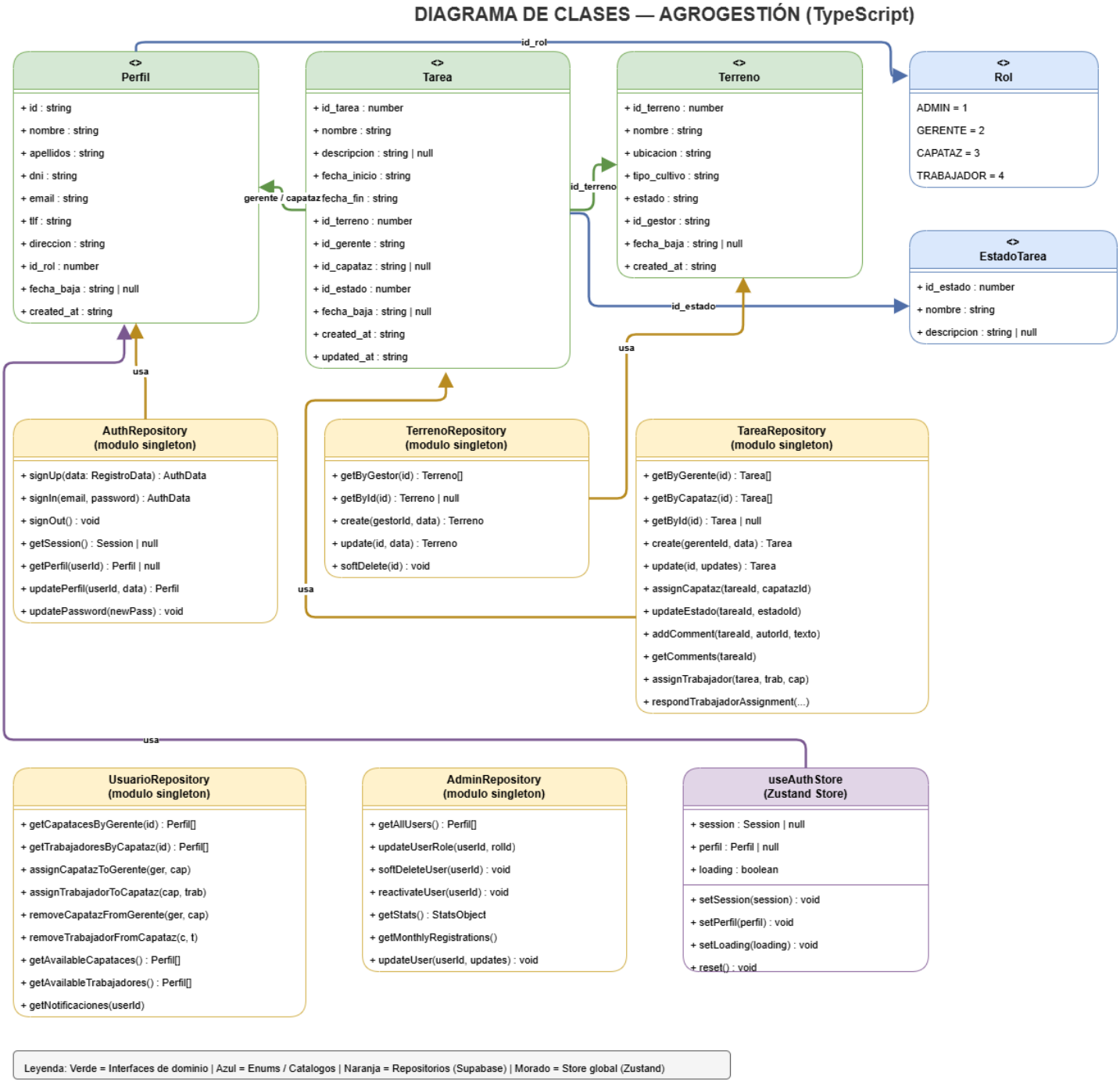
Diagrama de Clases — Agrogestión

1. Introducción

El presente documento describe el diagrama de clases UML del sistema **Agrogestión**, una aplicación web SPA (Single Page Application) desarrollada con **React 19**, **TypeScript** y **Supabase** como backend. El diagrama refleja la arquitectura del código fuente organizada en interfaces de dominio, repositorios de acceso a datos y un store global de estado.



2. Estructura del diagrama

El diagrama se organiza en tres filas por colores:

Color	Categoría	Elementos
Verde	Interfaces de dominio	Perfil, Tarea, Terreno
Azul	Enumeraciones / Catálogos	Rol (enum), EstadoTarea
Naranja	Repositorios (acceso a datos)	AuthRepository, TerrenoRepository, TareaRepository, UsuarioRepository, AdminRepository
Morado	Store global (estado de la app)	useAuthStore (Zustand)

3. Interfaces de dominio

3.1. Perfil

Representa el perfil de un usuario del sistema. Se corresponde con la tabla perfiles de la base de datos.

Atributo	Tipo	Descripción
id	string	UUID del usuario (PK, enlaza con auth.users)
nombre	string	Nombre del usuario
apellidos	string	Apellidos del usuario
dni	string	Documento Nacional de Identidad (9 caracteres)
email	string	Correo electrónico único
tlf	string	Número de teléfono
direccion	string	Dirección postal
id_rol	number	FK hacia el enum Rol
fecha_baja	string | null	Fecha de baja lógica (soft delete)
created_at	string	Fecha de creación del registro

3.2. Tarea

Representa una tarea agrícola asignada a un terreno, creada por un gerente y opcionalmente asignada a un capataz.

Atributo	Tipo	Descripción
id_tarea	number	Identificador único de la tarea
nombre	string	Nombre descriptivo de la tarea
descripcion	string | null	Descripción detallada
fecha_inicio	string	Fecha de inicio planificada
fecha_fin	string	Fecha de fin planificada
id_terreno	number	FK hacia Terreno

Atributo	Tipo	Descripción
id_gerente	string	FK hacia Perfil (gerente creador)
id_capataz	string | null	FK hacia Perfil (capataz asignado)
id_estado	number	FK hacia EstadoTarea
fecha_baja	string | null	Soft delete
created_at	string	Fecha de creación
updated_at	string	Última modificación
comentarios_tarea	{ count: number }[]	Contador de comentarios (campo virtual, cargado en la consulta SELECT)

3.3. Terreno

Representa una parcela o terreno de cultivo gestionado por un gerente.

Atributo	Tipo	Descripción
id_terreno	number	Identificador único del terreno
nombre	string	Nombre del terreno
ubicacion	string	Ubicación geográfica
tipo_cultivo	string	Tipo de cultivo plantado
estado	string	Estado actual del terreno
id_gestor	string	FK hacia Perfil (gerente propietario)
fecha_baja	string | null	Soft delete
created_at	string	Fecha de creación

4. Enumeraciones y catálogos

4.1. Rol (enum)

Define los 4 roles jerárquicos del sistema:

Valor	Código	Permisos principales
ADMIN	1	Gestión completa de usuarios y estadísticas
GERENTE	2	Gestión de terrenos, tareas y equipos de capataces
CAPATAZ	3	Gestión de tareas asignadas y equipos de trabajadores
TRABAJADOR	4	Visualización de tareas asignadas, aceptar/rechazar

4.2. Estado Tarea

Catálogo de los posibles estados de una tarea:

Atributo	Tipo	Descripción
id_estado	number	Identificador del estado
nombre	string	Nombre del estado (PENDIENTE, ASIGNADA, etc.)
descripcion	string | null	Descripción del estado

5. Capa de repositorios

Los repositorios encapsulan toda la lógica de acceso a datos a través del cliente de **Supabase**. Se implementan como **módulos singleton** (funciones exportadas directamente, sin instanciación de clases), un patrón típico en aplicaciones TypeScript/React modernas.

5.1. AuthRepository (7 métodos)

Gestiona la autenticación y el perfil del usuario autenticado.

Método	Retorno	Descripción
signUp(data)	AuthData	Registra un nuevo usuario en Supabase Auth
signIn(email, password)	AuthData	Inicia sesión con email y contraseña
signOut()	void	Cierra la sesión actual
getSession()	Session | null	Obtiene la sesión JWT vigente
getPerfil(userId)	Perfil | null	Carga el perfil desde la tabla perfiles
updatePerfil(userId, data)	Perfil	Actualiza los datos del perfil
updatePassword(newPass)	void	Cambia la contraseña del usuario

5.2. TerrenoRepository (5 métodos)

CRUD completo para la gestión de terrenos.

Método	Retorno	Descripción
getByGestor(id)	Terreno[]	Lista los terrenos de un gerente
getById(id)	Terreno | null	Obtiene un terreno por su ID
create(gestorId, data)	Terreno	Crea un nuevo terreno
update(id, data)	Terreno	Actualiza un terreno existente
softDelete(id)	void	Marca el terreno como dado de baja

5.3. TareaRepository (11 métodos)

El repositorio más complejo, gestiona todo el ciclo de vida de las tareas.

Método	Retorno	Descripción
getByGerente(id)	Tarea[]	Tareas creadas por un gerente
getByCapataz(id)	Tarea[]	Tareas asignadas a un capataz
getById(id)	Tarea | null	Obtiene una tarea por ID
create(gerenteId, data)	Tarea	Crea una nueva tarea
update(id, updates)	Tarea	Actualiza una tarea
assignCapataz(tareaId, capatazId)	—	Asigna un capataz a la tarea
updateEstado(tareaId, estadoId)	—	Cambia el estado de la tarea
addComment(tareaId, autorId, texto)	—	Añade un comentario a la tarea
getComments(tareaId)	Comentario[]	Lista los comentarios de una tarea
assignTrabajador(tarea, trab, cap)	—	Asigna un trabajador a una tarea
respondTrabajadorAssignment(...)	—	El trabajador acepta o rechaza la asignación

5.4. UsuarioRepository (10 métodos)

Gestiona las relaciones jerárquicas entre usuarios (equipos).

Método	Retorno	Descripción
getCapatacesByGerente(id)	Perfil[]	Capataces asignados a un gerente
getTrabajadoresByCapataz(id)	Perfil[]	Trabajadores asignados a un capataz
assignCapatazToGerente(ger, cap)	—	Asocia un capataz a un gerente
assignTrabajadorToCapataz(cap, trab)	—	Asocia un trabajador a un capataz
removeCapatazFromGerente(ger, cap)	—	Desasocia un capataz de un gerente
removeTrabajadorFromCapataz(c, t)	—	Desasocia un trabajador de un capataz
getAvailableCapataces()	Perfil[]	Capataces no asignados a ningún gerente
getAvailableTrabajadores()	Perfil[]	Trabajadores no asignados a ningún capataz
getNotificaciones(userId)	Notificación[]	Notificaciones del usuario
markAllNotificacionesLeidas(userId)	void	Marca todas las notificaciones como leídas

5.5. AdminRepository (7 métodos)

Funciones exclusivas del rol Administrador.

Método	Retorno	Descripción
getAllUsers()	Perfil[]	Lista todos los usuarios del sistema
updateUserRole(userId, rolId)	—	Cambia el rol de un usuario
softDeleteUser(userId)	void	Da de baja lógica a un usuario
reactivateUser(userId)	void	Reactiva un usuario dado de baja
getStats()	StatsObject	Estadísticas generales del dashboard
getMonthlyRegistrations()	RegistroMensual[]	Datos del gráfico de registros mensuales
updateUser(userId, updates)	void	Actualiza los datos de cualquier usuario

6. Store global — useAuthStore (Zustand)

El store global gestiona el estado de autenticación de la aplicación usando **Zustand**, una librería de gestión de estado ligera para React.

Estado (propiedades reactivas)

Propiedad	Tipo	Descripción
session	Session | null	Sesión JWT de Supabase
perfil	Perfil | null	Perfil del usuario autenticado
loading	boolean	Indicador de carga

Acciones (métodos)

Método	Descripción
setSession(session)	Actualiza la sesión en el store
setPerfil(perfil)	Actualiza el perfil en el store
setLoading(loading)	Cambia el estado de carga
reset()	Limpia todo el estado (logout)

7. Relaciones entre elementos

- **Perfil** referencia a **Rol** (a través de id_rol).
- **Tarea** referencia a **Perfil** (gerente y capataz), **Terreno** y **EstadoTarea**.
- **AuthRepository** usa la interfaz **Perfil** (lectura y escritura).
- **TerrenoRepository** usa la interfaz **Terreno**.
- **TareaRepository** usa la interfaz **Tarea**.
- **useAuthStore** almacena una instancia de **Perfil** como estado global.
- **UsuarioRepository** y **AdminRepository** trabajan con la interfaz **Perfil**.

8. Patrones de diseño aplicados

1. **Repository Pattern**: Los repositorios encapsulan el acceso a Supabase, desacoplando la lógica de negocio de la capa de datos.
2. **Singleton Module**: Cada repositorio es un módulo con funciones exportadas directamente (sin new), garantizando una única instancia del cliente Supabase.
3. **Store Pattern (Zustand)**: Estado global reactivo sin la complejidad de Redux, con actualizaciones inmutables automáticas.
4. **Soft Delete**: Eliminación lógica mediante fecha_baja en lugar de DELETE físico.